Gradient Descent in Neural Networks: Derivation, Empirical Verification and Convergence

Neel Kochhar

March 29, 2026

1 Introduction

In recent years, neural networks have been embedded as the cornerstone in the tech world. From their use in large language models such as OpenAI's ChatGPT or Google's Gemini, to usage in computer vision and robotics, they have become increasingly important. Neural networks are able to learn representations of data so that they can comprehend and utilize it as needed. However, training a neural network for many is not a trivial task, and the underlying training processes are often treated as a black box.

As I have recently begun my journey into learning machine learning, I have built a variety of simple projects, but no matter how deeply I understood the model architecture, I could not wrap my head around how the model was actually being trained. Although buzzwords such as backpropagation and gradient-descent showed up consistently, most textbooks did not dive deep into these topics. This paper investigates the training methodology used in neural networks. The research question is *how does gradient descent minimize the loss of a neural network, and under what conditions does it converge*? The theoretical derivation will also be empirically verified through the XOR dataset.

2 Mathematical Preliminaries

This exploration extends beyond the course curriculum of Math HL AA, including concepts regarding multivariate functions. A multivariate function is a function that depends on two or more variables such as $f(x, y) = x^3y^2 - 2xy$ (Stewart et al., 2020). The following concepts of partial derivatives and chain rule for multivariate functions are introduced below to aid in deriving gradient descent.

2.1 Partial Derivatives

A partial derivative deals with multivariate functions. It measures how a multivariate function varies with respect to a single variable while the other variables are treated as constant (Khan Academy, 2024). The same rules for normal derivatives naturally follow.

Suppose

$$f(x, y) = x^3y^2 - 2xy$$

Using the power rule,

$$\frac{\partial f}{\partial x} = 3x^2y^2 - 2y$$

$$\frac{\partial f}{\partial y} = 2yx^3 - 2x$$

Suppose that we wanted to take the partial derivatives for the above function at the point (3,4),

$$\left.\frac{\partial f}{\partial x}\right|_{3,4} = 3(3)^2 4^2 - 2(4) = 424$$

$$\left.\frac{\partial f}{\partial y}\right|_{3,4} = 2(4)(3)^3 - 2(3) = 210$$

(Khan Academy, 2024)

2.2 Chain Rule for Multivariate Functions

Recall that to differentiate a single variable function that depends on another function $f(g(x))$, we use the chain rule,

$$\frac{df}{dx} = \frac{df}{dg} \cdot \frac{dg}{dx}$$

The chain rule naturally extends to a multivariable context. Let us suppose A depends on B which depends on C which depends on $x_1, x_2, ..., x_n$. To isolate one variable (say x_n) at a time, we use partial derivatives.

$$\frac{\partial A}{\partial x_n} = \frac{\partial A}{\partial B} \cdot \frac{\partial B}{\partial C} \cdot \frac{\partial C}{\partial x_n}$$

(Stewart et al., 2020)

2.3 Gradient

The gradient is a vector of all the partial derivatives for a function (Stewart et al., 2020). For the function $f(x, y)$ described above,

$$\nabla f = (\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}) = (3x^2y^2 - 2y, 2yx^3 - 2x)$$

This gradient is a vector that points towards the steepest slope of increase for the function. A derivative is just an application of a gradient applied to a one-dimensional function.

3 Neural Network Overview

A neural network is simply a model that takes an input x and produces a prediction output $\hat{y} = f(x)$. The output depends on the current *state* of the model; or its parameters. Therefore, it can be stated that for any input x the model outputs $\hat{y} = f(x \mid \theta)$ where θ represents the model's parameters. The prediction output $\hat{y}$

is compared with the *label*, the *correct* match for the original input. The deviation from the label y and $\hat{y}$ is used to change the parameters of the model, hoping to minimize the deviation in the future. (Prince, 2023).

3.1 Architecture

A neural network is separated into multiple layers, composed of neurons. In a fully connected neural network, each neuron is connected to the neurons in the next layer, passing their output as their input. The figure below depicts a simple neural network and the architecture that will be used for this paper.

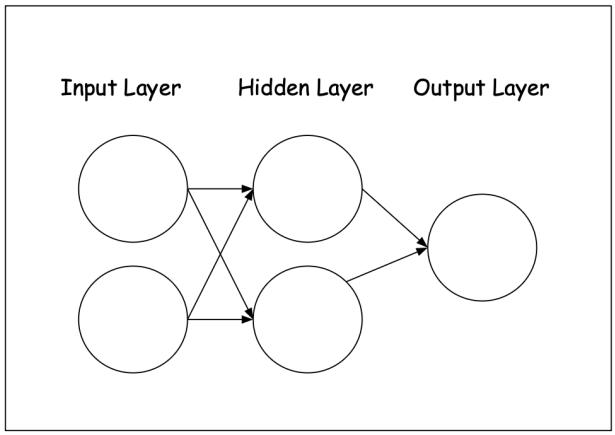

Figure 1: Simple Neural Network

The first layer, the *input layer*, is made up of two neurons. This means that the input data $x \in \mathbb{R}^2$. After being processed from the input layer, the output flows to the *hidden layer* which has two neurons. Finally, the output from the hidden layer flows to the *output layer*. This layer has a single neuron, so the output is one dimensional. Each neuron also applies a non-linear function at the end called an *activation function* to its input. In this paper, the popular sigmoid function is used, which maps any real number to the range (0,1). This function is discussed in detail in the following section. Without non-linearity that this function introduces, the network would reduce to a linear transformation (see section 5.1), making it impossible to learn complex relationships in data (Prince, 2023).

3.2 Forward Propagation

Forward propagation is the process of transforming the input x to prediction $\hat{y}$. Each layer is computed as a whole using a weight matrix. The weight matrix W contains the learnable parameters that connect the one layer to the next layer (Raff, 2022). The matrix entry, $W_{i,j}$, represents the weight from neuron j to i. Visually, in the above example, the connecting lines are the weights, and they transform input from one form to the next. A bias term is also summed onto the layer output to shift the pre-activation ouput independently of the input. This results in a pre-activation layer output z. As hinted, this z is passed through the activation function to achieve the final layer output h. The activation function in this paper is sigmoid (Prince, 2023):

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Overall, the forward propagation equations for this model are,

$$z_1 = W^{(1)}x + b_1, \quad W^{(1)} \in \mathbb{R}^{2\times 2},\ b_1 \in \mathbb{R}^2 \tag{1}$$

$$h_1 = \sigma(z_1), \quad h_1 \in (0,1) \tag{2}$$

$$z_2 = W^{(2)}h_1 + b_2, \quad W^{(2)} \in \mathbb{R}^{1\times 2},\ b_2 \in \mathbb{R} \tag{3}$$

$$\hat{y} = \sigma(z_2), \quad \hat{y} \in (0,1) \tag{4}$$

where $\hat{y}$ is the model's final prediction for the column vector x with shape (2×1)(Raff, 2022).

3.3 Model Loss

The last part of the neural network that is essential to understand is the concept of loss. The data on which the model is being trained on comes in pairs: (x, y). Each input comes paired with a label, the corresponding answer for the data. The model generates its prediction for the input y. The concept of loss is the deviation between y and $\hat{y}$. There are different measures of this deviation, and the one being used in this paper is mean squared error (MSE). Defined as,

$$L = (y - \hat{y})^2 \tag{5}$$

This MSE loss measures the squared deviation between the prediction and the label (Prince, 2023). Additionally, I think the reason that the difference is squared rather than taken as an absolute value is that the loss function is differentiable everywhere. Furthermore, the goal of the neural network is to minimize the loss function with respect to θ - (collection of all trainable weights and trainable biases). This is achieved through gradient descent, analyzing which is the motivation behind this paper.

3.4 Notation

The following notation will be used throughout the paper.

Table 1: Notation Table

Symbol	Description
x	Input vector
y	True label
$\hat{y}$	Prediction output
$W^{(1)}, W^{(2)}$	Weight matrices
b_1, b_2	Biases
z_1, z_2	Pre-activation output
h_1	Post-activation output
σ	Activation function
L	MSE Loss
η	Learning rate
θ	Collection of all trainable parameters (weights and biases)

4 Derivation of Gradient Descent

As per any calculus optimization problem, the goal is to allow the function to reach a minimum point. This is achieved by traversing along the negative gradient, pointing in the steepest slope of descent. Naturally, the first step is to compute the gradient of the function that we are trying to minimize; in this case, the MSE loss L. This process of computing the derivatives is known as backpropagation. Once the derivatives, and hence gradients, are found, they are used to update the model. This is known as gradient descent (Nielsen, 2015) where the original weight matrix and biases are replaced by,

$$W \leftarrow W - \eta \nabla_W L, \quad \nabla_W L = \left(\frac{\partial L}{\partial W^{(2)}}, \frac{\partial L}{\partial W^{(1)}} \right)$$

$$b \leftarrow b - \eta \nabla_b L, \quad \nabla_W L = \left(\frac{\partial L}{\partial b_2}, \frac{\partial L}{\partial b_1} \right)$$

η is a scalar value that controls the percentage of the gradient we traverse by. Too high, and that can lead to drastic changes. Conversely, too little, and it will take longer to reach the local minima. The value required for the learning rate will be discussed in section 5, convergence analysis.

4.1 Sigmoid Derivative

When I was calculating the derivative for the sigmoid activation function, I felt that my process deserved its own subsection. Firstly, for simplicity, I rewrote sigmoid without the fraction to avoid the quotient rule, for a simpler derivation,

$$\sigma(x) = \frac{1}{1+e^{-x}} = (1+e^{-x})^{-1}$$

Then to compute the derivative, using chain rule and power rule,

$$\begin{aligned} \frac{d}{dx}(1+e^{-x})^{-1} &= -(1+e^{-x})^{-2} \cdot \frac{d}{dx}(1+e^{-x}), \\ &= -(1+e^{-x})^{-2} \cdot (0-e^{-x}), \\ &= \frac{e^{-x}}{(1+e^{-x})^2} \end{aligned}$$

To simplify, I thought to split the denominator as I could see how to plug the sigmoid function back in.

$$\begin{aligned} \frac{e^{-x}}{(1+e^{-x})^2} &= e^{-x} \cdot \frac{1}{1+e^{-x}} \cdot \frac{1}{1+e^{-x}} \\ &= e^{-x} \cdot \sigma(x)^2 \end{aligned}$$

Although the result is clean enough to employ in the next sections, I learned the derivative could be written more elegantly, all in terms of sigmoid (Nielsen, 2015). I wanted to find the steps myself, and luckily, it did

not take long, as I was close previously. All I needed was some algebraic manipulation,

$$\begin{aligned}\frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}} &= \sigma(x) \cdot \frac{e^{-x}+1-1}{1+e^{-x}} \\ &= \sigma(x) \cdot \left(\frac{1+e^{-x}}{1+e^{-x}} - \frac{1}{1+e^{-x}}\right) \\ \sigma'(x) &= \sigma(x)(1-\sigma(x)) \end{aligned} \tag{6}$$

4.2 Output Layer Derivatives

As described, the goal of backpropagation is to compute the gradient for the loss function. To build the gradient, first I needed to find the partial derivatives. As per chain rule, I worked from the outside function inwards. As the output layer acts as an outside function to the inner layer, I computed the partial derivatives for the output layer first. These were $\frac{\partial L}{\partial W^{(2)}}$ and $\frac{\partial L}{\partial b_2}$. Using equations (3), (4) from section 3.2, and (5) from section 3.3, I started to build the expressions for both derivatives. Using chain rule,

$$\begin{aligned}\frac{\partial L}{\partial W^{(2)}} &= \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} \cdot \frac{\partial z_2}{\partial W^{(2)}} \\ &= 2(y-\hat{y}) \cdot \frac{\partial}{\partial \hat{y}}(y-\hat{y}) \cdot \frac{\partial \hat{y}}{\partial z_2} \cdot \frac{\partial z_2}{\partial W^{(2)}} \\ &= 2(y-\hat{y}) \cdot (-1) \cdot \frac{\partial \hat{y}}{\partial z_2} \cdot \frac{\partial z_2}{\partial W^{(2)}} \\ &= -2(y-\hat{y}) \cdot \sigma'(z_2) \cdot \frac{\partial z_2}{\partial W^{(2)}}\end{aligned}$$

Applying the sigmoid derivative from equation (6),

$$\begin{aligned}\frac{\partial L}{\partial W^{(2)}} &= -2(y-\hat{y}) \cdot \sigma(z_2)(1-\sigma(z_2)) \cdot \frac{\partial z_2}{\partial W^{(2)}} \\ &= -2(y-\hat{y}) \cdot \sigma(z_2)(1-\sigma(z_2)) \cdot h_1\end{aligned}$$

But, as stated in equation (4), $\hat{y}_1 = \sigma(z_2)$, the above can be simplified into,

$$\frac{\partial L}{\partial W^{(2)}} = -2(y-\hat{y}) \cdot \hat{y}(1-\hat{y}) \cdot h_1 \tag{7}$$

A similar derivation follows for the bias term. The only difference is that the last derivative in the chain, $\frac{\partial z_2}{\partial b_2} = 1$. Following the same chain rule expansion as above, I arrived at,

$$\begin{aligned}\frac{\partial L}{\partial b_2} &= \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} \cdot \frac{\partial z_2}{\partial b_2} \\ &= -2(y-\hat{y}) \cdot \hat{y}(1-\hat{y}) \cdot \frac{\partial z_2}{\partial b_2} \\ &= -2(y-\hat{y}) \cdot \hat{y}(1-\hat{y})\end{aligned} \tag{8}$$

4.3 Hidden Layer Derivatives

Now that I had the derivatives for the output layer, namely, $\frac{\partial L}{\partial W^{(2)}}$ and $\frac{\partial L}{\partial b_2}$, the next part was to find the hidden layer derivatives. After, I could move onto creating the final update rule. Using the chain rule and equations (1), (2), (3) (4) and (5) from section 3.2 and 3.3,

$$\frac{\partial L}{\partial W^{(1)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z_2} \cdot \frac{\partial z_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial W^{(1)}}$$

However, when I was solving the derivative above, I realized the first two parts of the chain expansion belonged to the hidden layer. I had already computed this derivative previously. From section 4.2, equation (7) contains the partial derivative $\frac{\partial L}{\partial z_2}$. That is,

$$\frac{\partial L}{\partial z_2} = -2(y - \hat{y}) \cdot \hat{y}(1 - \hat{y})$$

Therefore, the derivative chain for the hidden layer with respect to the weight matrix could be expressed as,

$$\begin{aligned}
\frac{\partial L}{\partial W^{(1)}} &= \frac{\partial L}{\partial z_2} \cdot \frac{\partial z_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial W^{(1)}} \\
&= -2(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}) \cdot \frac{\partial z_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial W^{(1)}} \\
&= -2(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}) \cdot W^{(2)} \cdot \frac{\partial h_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial W^{(1)}} \\
&= -2(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}) \cdot W^{(2)} \cdot \sigma'(z_1) \cdot \frac{\partial z_1}{\partial W^{(1)}} \\
&= -2(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}) \cdot W^{(2)} \cdot \sigma(z_1)(1 - \sigma(z_1)) \cdot \frac{\partial z_1}{\partial W^{(1)}} \\
&= -2(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}) \cdot W^{(2)} \cdot \sigma(z_1)(1 - \sigma(z_1)) \cdot x
\end{aligned}$$

Again, since $h_1 = \sigma(z_1)$, I simplified the expression like,

$$\frac{\partial L}{\partial W^{(1)}} = -2(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}) \cdot W^{(2)} \cdot h_1 \cdot (1 - h_1) \cdot x \tag{9}$$

Similarly, the loss derivative with respect to the bias term in the hidden layer can be computed as,

$$\begin{aligned}
\frac{\partial L}{\partial b_1} &= \frac{\partial L}{\partial z_2} \cdot \frac{\partial z_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial b_1} \\
&= -2(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}) \cdot W^{(2)} \cdot h_1 \cdot (1 - h_1) \cdot \frac{\partial z_1}{\partial b_1} \\
\frac{\partial L}{\partial b_1} &= -2(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}) \cdot W^{(2)} \cdot h_1 \cdot (1 - h_1)
\end{aligned} \tag{10}$$

Before I move on to the update rule, I realized that a big portion of this algorithm is how the derivatives are reused. In my specific architecture, the $\frac{\partial L}{\partial z_2}$ was calculated in the output layer, but then reused in the hidden layer. I believe that the more layers there are in the model, the more useful this reusing will prove; cutting down on calculation costs for the backpropagation part of gradient descent.

4.4 Update Rule

Now that I have the derivatives, I can compile the gradients for the loss function with respect to the weight matrix, and the biases. From here, I can write out the explicit update rule for this model architecture chosen. Firstly, I wrote out the gradients,

$$\nabla_W L = \left(\frac{\partial L}{\partial W^{(2)}}, \frac{\partial L}{\partial W^{(1)}} \right)$$

$$\nabla_b L = \left(\frac{\partial L}{\partial b_2}, \frac{\partial L}{\partial b_1} \right)$$

Subbing in equations (7), (9) into the weight gradient, and equations (8) and (10) into the bias gradient,

$$\nabla_W L = \Big(-2(y-\hat{y}) \cdot \hat{y}(1-\hat{y}) \cdot h_1, \;\; -2(y-\hat{y}) \cdot \hat{y}(1-\hat{y}) \cdot W^{(2)} \cdot h_1(1-h_1) \cdot x \Big)$$

$$\nabla_b L = \Big(-2(y-\hat{y}) \cdot \hat{y}(1-\hat{y}), \;\; -2(y-\hat{y}) \cdot \hat{y}(1-\hat{y}) \cdot W^{(2)} \cdot h_1(1-h_1) \Big)$$

Now I could move on to the update rules. For these, I realized that since W_1 and b_1 update depends on W_2, I have to update W_1 and b_1 first. I found this interesting. Initially, the input propagates forward from the input to output layer. Then I use backpropagation going from output to input to calculate the gradient. To update, now I go from input to output. However, thinking mathematically, this process is clear, due to the nature of the chain rule. This is so to ensure the correct dependency chain. Hence, he update rules goes as follows,

$$W^{(1)} \leftarrow W^{(1)} + 2\eta(y-\hat{y}) \cdot \hat{y}(1-\hat{y}) \cdot W^{(2)} \cdot h_1(1-h_1) \cdot x \tag{11}$$

$$b_1 \leftarrow b_1 + 2\eta(y-\hat{y}) \cdot \hat{y}(1-\hat{y}) \cdot W^{(2)} \cdot h_1(1-h_1) \tag{12}$$

$$W^{(2)} \leftarrow W^{(2)} + 2\eta(y-\hat{y}) \cdot \hat{y}(1-\hat{y}) \cdot h_1 \tag{13}$$

$$b_2 \leftarrow b_2 + 2\eta(y-\hat{y}) \cdot \hat{y}(1-\hat{y}) \tag{14}$$

5 Application to XOR

5.1 Motivation Through Non-Linearity

XOR is a popular dataset in statistics and machine learning to solve problems that can not be solved with a linear model. Before I applied this to my neural network, I wanted to understand why this problem can not be achieved through a linear result. Suppose I have a linear model where the state of the model is composed of the following parameters, w_1, w_2, b, where w_1, w_2 are two weights. This model has a single layer. The XOR dataset has only 4 data points. These are,

Table 2: XOR Dataset

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

According to my model, each x_1 is multiplied with w_1, and each x_2 is multiplied with w_2. Since the output is binary, if it is less than 0, it will map to 0, and conversely, if it is more than 0, I will map it to 1. So, the data is processed as,

$$\begin{cases} w_1(0) + w_2(0) + b & < 0, \\ w_1(0) + w_2(1) + b & > 0, \\ w_1(1) + w_2(0) + b & > 0, \\ w_1(1) + w_2(1) + b & < 0, \end{cases}$$

This simplifies into,

$$\begin{cases} b < 0 \\ w_2 + b > 0 \\ w_1 + b > 0 \\ w_1 + w_2 + b < 0 \end{cases}$$

Adding the middle two equations together, and the remaining others together, I arrived at, $\begin{cases} w_1 + w_2 + 2b > 0 \\ w_1 + w_2 + 2b < 0 \end{cases}$, which raises a contradiction. No w_1, w_2, b exists to satisfy this system. Therefore, it is proved that no linear model can solve the dataset. I had purposefully chosen my model; represented through equations (1) to (5), to be able to solve this dataset. It is able to do so due to the multiple layers coupled with activation functions. Without an activation function, the presence of multiple layers would just reduce down to a linear combination: $W_1(W_2^T(x)) = (W_1 W_2^T)x$. Note that the transposition required due to the shapes of the weight matrices (outlined in equations (1) and (3)). However, with the presence of an activation function, the model is no longer able to be reduced down to a linear combination as shown above. With this, the model can solve non-linear problems.

5.2 Empirical Run Through

First, I wanted to run through the gradient descent update rule once for a data point, and see if the loss decreased for that point; just to see empirical evidence of my calculations being correct. Afterwards, I wrote a program in Python to test on the dataset for a full training. In machine learning, the initialization of the weights are randomized. I initialized the values to be the following,

$$W^{(1)} = \begin{bmatrix} 0.5 & 0.2 \\ 0.3 & 0.8 \end{bmatrix}, \quad W^{(2)} = \begin{bmatrix} 0.6 & 0.4 \end{bmatrix}, \quad b_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad b_2 = \begin{bmatrix} 0 \end{bmatrix}$$

The learning rate I used was $\eta = 0.1$ and I chose to do this empirical verification on the data point

$$x = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad y = \begin{bmatrix} 1 \end{bmatrix}$$

Passing the data through the model equations (2) to (4),

$$z_1 = W^{(1)}x + b_1 = \begin{bmatrix} 0.5 & 0.2 \\ 0.3 & 0.8 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.2 \\ 0.8 \end{bmatrix}$$

$$h_1 = \sigma(z_1) = \begin{bmatrix} \sigma(0.2) \\ \sigma(0.8) \end{bmatrix} = \begin{bmatrix} 0.550 \\ 0.690 \end{bmatrix}$$

$$z_2 = W^{(2)}h_1 + b_2 = \begin{bmatrix} 0.6 & 0.4 \end{bmatrix} \begin{bmatrix} 0.550 \\ 0.690 \end{bmatrix} + [0] = 0.606$$

$$\hat{y} = \sigma(z_2) = \sigma(0.606) = 0.647$$

Using equation (5), the current mean squared error loss is,

$$L = (y - \hat{y})^2 = (1 - 0.647)^2 = 0.125 \tag{15}$$

Now I used the update rules (11) to (14) to update the state of the model. W_1 is updated as,

$$W^{(1)} \leftarrow W^{(1)} + 2\eta(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}) \cdot W^{(2)} \cdot h_1(1 - h_1) \cdot x$$

$$W^{(1)} \leftarrow \begin{bmatrix} 0.5 & 0.2 \\ 0.3 & 0.8 \end{bmatrix} + 2(0.1)(1 - 0.647) \cdot 0.647(1 - 0.647) \cdot \begin{bmatrix} 0.6 & 0.4 \end{bmatrix} \cdot \begin{bmatrix} 0.550 \\ 0.690 \end{bmatrix} \left(1 - \begin{bmatrix} 0.550 \\ 0.690 \end{bmatrix} \right) \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$W^{(1)} \leftarrow \begin{bmatrix} 0.5 & 0.202 \\ 0.3 & 0.801 \end{bmatrix}$$

Similarly, the other parameters are adjusted as,

$$b_1 \leftarrow \begin{bmatrix} 0.002 \\ 0.001 \end{bmatrix}$$

$$W^{(2)} \leftarrow \begin{bmatrix} 0.609 & 0.411 \end{bmatrix}$$

$$b_2 \leftarrow [0.016]$$

With the new state of the model, I calculated $\hat{y}^{'} = 0.650$. Then I calculated the new loss,

$$L^{'} = (y\_\hat{y})^2 = (1 - 0.650)^2 = 0.120 \tag{16}$$

Comparing the initial loss (15) with the loss post-update (16), I saw that the loss went down. This shows empirical support for my gradient descent equations by showing how the model

6 Convergence Analysis

7 Reflection

8 Conclusion

References

Khan Academy. (2024). *Introduction to partial derivatives*. https://www.khanacademy.org/math/multivariable-calculus/multivariable-derivatives/partial-derivative-and-gradient-articles/a/introduction-to-partial-derivatives

Nielsen, M. A. (2015). *Neural networks and deep learning* (Vol. 25). Determination press San Francisco, CA, USA.

Prince, S. J. (2023). *Understanding deep learning*. MIT press.

Raff, E. (2022). *Inside deep learning: Math, algorithms, models*. Simon; Schuster.

Stewart, J., Clegg, D. K., & Watson, S. (2020). *Multivariable calculus*. Cengage Learning.

A NumPy Model Implementation

```
class SimpleModel:
    def __init__(self):
        self.W1 = np.array([[0.5, 0.2], [0.3, 0.8]])
        self.b1 = np.zeros((2,1))
        self.W2 = np.array([[0.6, 0.4]])
        self.b2 = np.zeros((1, 1))
        self.lr = 50
        self.y_hat = 0
        self.x = np.zeros((2,1))

    def forward(self, x):
        self.x = x # (2x1)
        self.z1 = self.W1 @ x + self.b1
        self.h1 = sigmoid(self.z1)
        self.z2 = self.W2 @ self.h1 + self.b2
        self.y_hat = sigmoid(self.z2)

        return self.y_hat

    def updateState(self, y):
        # delta = dL/dz2
        delta = 2 * self.lr * (y - self.y_hat) * self.y_hat * (1 - self.y_hat)
        self.W1 = self.W1 + delta * self.W2.T * self.h1 * (1 - self.h1) * self.x.T
        self.b1 = self.b1 + delta * self.W2.T * self.h1 * (1 - self.h1)
        self.W2 = self.W2 + delta * self.h1.T
        self.b2 = self.b2 + delta
```

B Decision Boundary Graph